

CS 550 Operating Systems

Spring 2026

Process API

Overview

- fork-exec model (Linux):
 - `fork()` creates a copy of the current process
 - `exec*()` replaces the current process' code and address space with the code of a different program
 - Family: `execv`, `execl`, `execve`, `execle`, `execvp`, `execlp`
 - `fork()` and `execve()` are C library functions. There are system calls corresponding to them
- Other API for process management:
 - `getpid()`
 - `exit()`
 - `wait()`, `waitpid()`

fork() : Creating new processes

- `pid_t fork(void)`
 - Creates a new “**child**” process that is ***almost identical*** to the calling “**parent**” process, including all the states (memory, registers, etc.)
 - Returns 0 to the **child** process
 - Returns child’s **process ID (PID)** to the **parent** process
- Child is ***almost identical*** to parent:
 - Child gets an identical (but separate) copy of the parent’s virtual address space
 - Child has a different PID than the parent

```
pid_t pid = fork();  
if (pid == 0) {  
    printf("hello from child\n");  
} else {  
    printf("hello from parent\n");  
}
```

`fork()` is unique (and often confusing) because it is called **once** but returns **“twice”**

Process X (parent)

Before
calling
fork()

```
...  
pid_t pid = fork();  
if (pid == 0) {  
    printf("hello from child\n");  
} else {  
    printf("hello from parent\n");  
}
```

In the
middle of
calling
fork()

```
...  
pid_t pid = fork();  
if (pid == 0) {  
    printf("hello from child\n");  
} else {  
    printf("hello from parent\n");  
}
```

After
calling
fork()

```
...  
pid_t pid = fork();  
if (pid == 0) {  
    printf("hello from child\n");  
} else {  
    printf("hello from parent\n");  
}
```

pid = Y

Process Y (child)

```
...  
pid_t pid = fork();  
if (pid == 0) {  
    printf("hello from child\n");  
} else {  
    printf("hello from parent\n");  
}
```

```
...  
pid_t pid = fork();  
if (pid == 0) {  
    printf("hello from child\n");  
} else {  
    printf("hello from parent\n");  
}
```

pid = 0

Let's look at this code ...

```
int main() {  
    fork();  
    fork();  
    printf("Hello!\n");  
    return 0;  
}
```

Q: How many times does “Hello!\n” get printed?

Fork Example

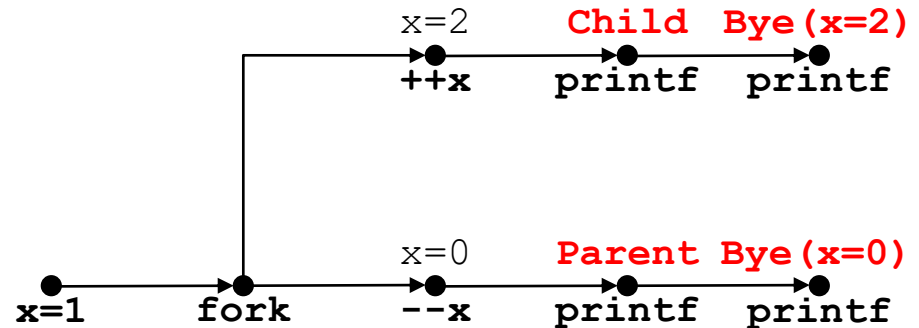
```
void main() {
    int x = 1;
    pid_t pid = fork();
    if (pid == 0)
        printf("Child has x = %d\n", ++x);
    else
        printf("Parent has x = %d\n", --x);
    printf("Bye from process %d with x = %d\n", getpid(), x);
}
```

- Both processes continue/start execution after `fork`
 - Child starts at instruction after the call to `fork` (storing into `pid`)
- Can't predict execution order of parent and child
- Both processes start with `x=1`
 - Subsequent changes to `x` are independent
- Shared open files: `stdout` is the same in both parent and child

Fork Example: Possible Output

```
void main() {  
    int x = 1;  
    pid_t pid = fork();  
    if (pid == 0)  
        printf("Child has x = %d\n", ++x);  
    else  
        printf("Parent has x = %d\n", --x);  
    printf("Bye from process %d with x = %d\n", getpid(), x);  
}
```

Since the execution order of the parent and the child processes is not deterministic, how many possible outputs does this program have?



```

void main() { // nested fork
    printf("L0\n");
    if (fork() == 0) {
        printf("L1\n");
        if (fork() == 0) {
            printf("L2\n");
        }
    }
    printf("Bye\n");
}

```

Q: Are the two output sequences on the right possible or not?

Seq 1:	Seq 2:
L0	L0
L1	Bye
Bye	L1
Bye	L2
Bye	Bye
L2	Bye

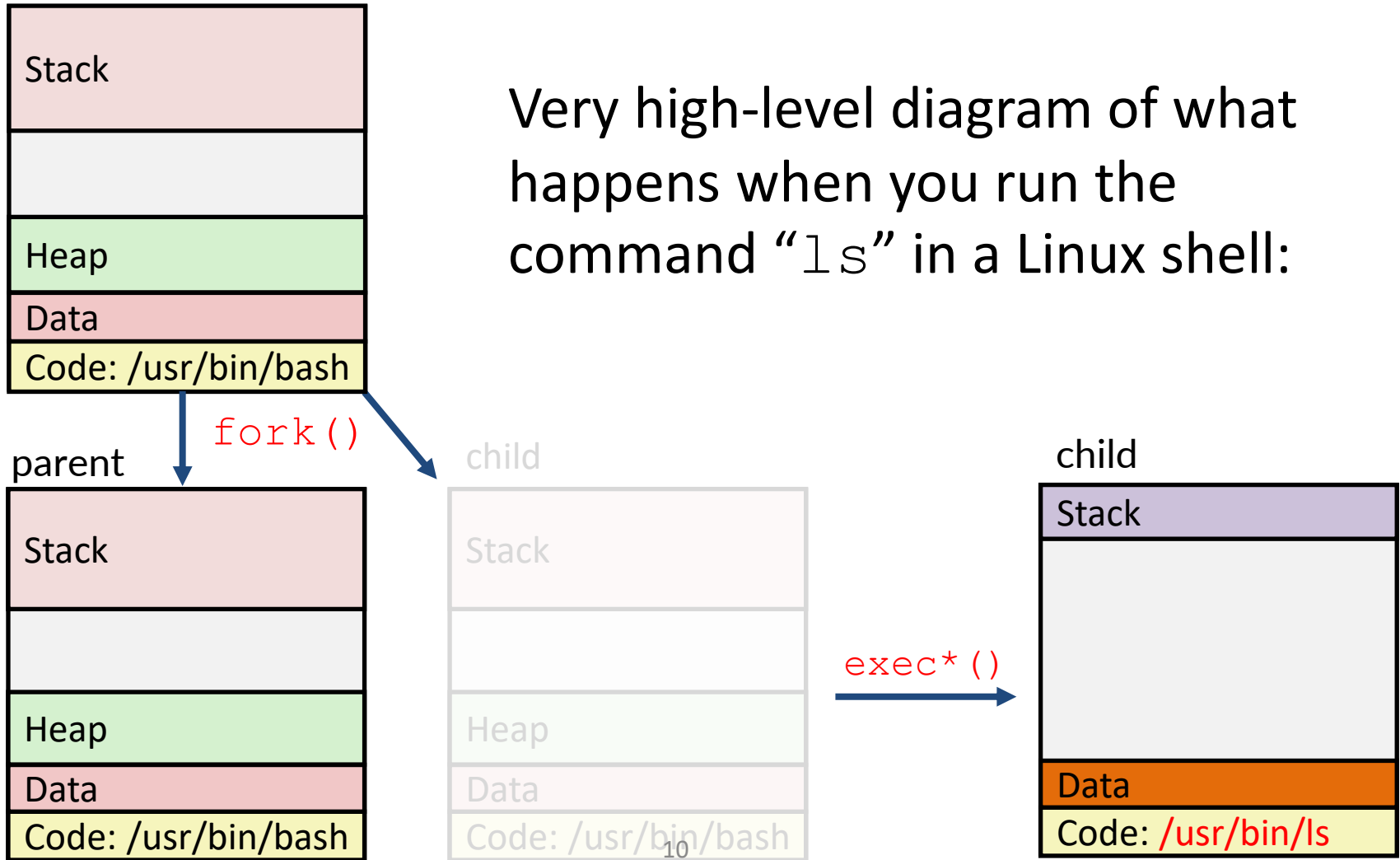
Fork-Exec

- fork-exec model:
 - `fork()` creates a copy of the current process
 - `exec*()` replaces the current process' code and address space with the code for a different program
 - Whole family of `exec` calls – see **`exec(3)`** and **`execve(2)`**

```
// Example arguments: path="/usr/bin/ls",  
//      argv[0]="/usr/bin/ls", argv[1]="-ahl", argv[2]=NULL  
  
void main(char *path, char *argv[]) { // fork-exec  
    pid_t pid = fork();  
    if (pid != 0) {  
        printf("Parent: created a child %d\n", pid);  
    } else {  
        printf("Child: about to exec a new program\n");  
        execv(path, argv);  
    }  
    printf("This line printed by parent only!\n");  
}
```

Exec-ing a new program

Very high-level diagram of what happens when you run the command “ls” in a Linux shell:



Terminating a process normally

- Three way to terminate a process normally
 - Calling `void exit(int status)`
 - Terminates the process (also flushes I/O buffers, executes exits functions registered by atexit(), ...)
 - Calling `void _exit(int status)`
 - Just terminates the process.
 - Return from the `main()` function.

Zombie & Orphan Processes

- When a process terminates, it still consumes system resources
 - Various tables maintained by OS
 - Called a “**zombie**” process (a living corpse, half alive and half dead)
 - The zombies are not removed from the OS because they contain valuable info that the parents may want to know.
 - What are the valuable info? Recall how the `exit()/_exit()` functions are called.
- *Reaping* should be performed by parent on terminated child
 - Parent obtain the **exit status information of the terminated child**, and then the OS kernel can totally remove the zombie child process from the system
- What if parent doesn't reap?
 - If any parent terminates without reaping a child, then the orphaned child will be reaped by a special process, for example, the `init` process (`pid == 1`)
 - **Note:** on more recent Linux systems, `init` has been renamed `systemd`
 - In long-running processes (e.g. shells, servers) we need *explicit* reaping

`wait`: Synchronizing with Children

`pid_t` `wait(int *child_exit_status)`

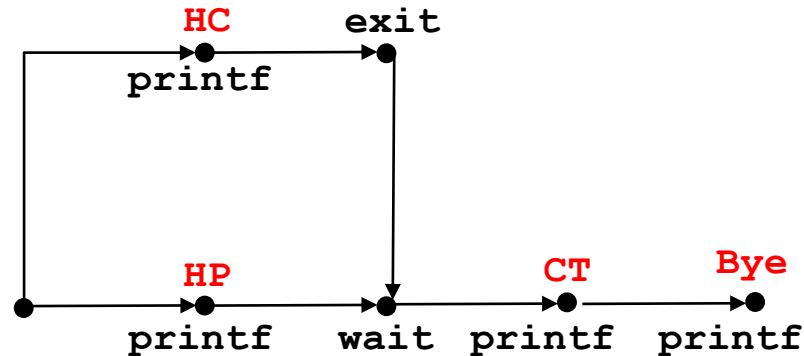
- Called by the parent process to reap one of this child processes
- Suspends the calling process (*i.e.* the parent) until **one of its children** terminates
- Return value is the PID of the child process that terminated, or -1 on error (e.g., no child proc is running).
 - *On successful return, the child process is reaped (*i.e.*, the terminated child process is totally removed from the system).*
- If `child_exit_status` is not NULL, then the `*child_status` value indicates why the child process terminated
 - Special macros for interpreting this status

Note: If parent process has multiple children, `wait` will return when *any* of the children terminates

- `waitpid` can be used to wait on a specific child process
 - `waitpid` can also specify different options for the wait, e.g., WNOHANG

wait: Synchronizing with Children

```
void fork_wait() {  
    int child_status;  
  
    if (fork() == 0) {  
        printf("HC: hello from child\n");  
        exit(0);  
    } else {  
        printf("HP: hello from parent\n");  
        wait(&child_status);  
        printf("CT: child has terminated\n");  
    }  
    printf("Bye\n");  
}
```



Process Management API Summary

- `fork` makes two copies of the same process (parent & child)
 - Returns different values to the two processes
- Two scenarios of using `fork`
 - Two-process program:
 - `pid = fork()`
`if (pid == 0) { /* child code */ } else { /* parent code */ }`
 - Two different programs:
 - `pid = fork()`
`if (pid == 0) { execv(...) } else { /* parent code */ }`
 - `exec*` replaces current process from file (new program)
- `wait` or `waitpid` used to synchronize parent/child execution and to reap child process